VOLUME 08 / 08

RESEARCH AND RELEASE

Turn code and experiments into an artifact someone can trust

Organize hypotheses, preserve evidence, reproduce reports, publish a clear repository and prepare a technical explanation.

Question → prediction → experiment → evidence

A hands-on companion to the RISC-V project. Examples, implementation steps, debugging and completion checks. Personal design edition / October 2026.

What you will build

You will keep a research trail across every build stage and publish a reproducible, accurately scoped project. The output is an evidence-backed repository and a demonstration you can explain.

Before you start

Use the earlier volumes for implementation. Do not assume a metric or experiment is complete because it appears in a plan.

Files and outputs

docs/EXPERIMENT_LOG.md, docs/RESEARCH.md, README.md, reports/, .github/workflows/verify.yml and your learning branch.

Build order

1. Write questions before editing.
2. Preserve exact code, inputs and commands.
3. Compare one variable at a time.
4. Reproduce in a clean environment.
5. Publish the complete evidence boundary.
6. Prepare a specific technical discussion.

The checked-in core is an assistant-generated reference. Work on a learning branch. The code excerpts explain selected regions and are not complete replacement modules unless explicitly labeled.

Step 1: start with a question you can test

A research question names a behavior or tradeoff and a comparison. “Is the core good?” is too broad. “Does this forwarding priority select the newest result?” has a direct counterexample test.

Write a hypothesis and an observation that would contradict it. Then select the smallest program or implementation run that could expose the difference.

For the pipeline, ask whether every dependent consumer sees the right producer. For DOT4, ask whether cycle savings survive clock/area costs. For verification, ask which bug could survive the current bin plan.

Use primary ISA/tool sources to establish definitions, and local experiments to establish this implementation's behavior. Do not cite a lab's research page as evidence that your own design is correct.

The project began from a placeholder README and an assistant-generated reference, with no earlier core draft. Preserve that provenance while recording the work you personally perform.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 8A. Turn “DOT4 is faster” into a falsifiable question with an explicit baseline and workload.

Step 2: spread research across the build

Foundations	Predict old/new values across an edge and test the prediction.
Datapath	Derive encodings and signed behavior from the ISA, then test edge cases.
Pipeline	Predict each stage's instruction identity and compare the trace.
Hazards	Disable one mechanism in an isolated experiment and identify its first failure.
DOT4	Compare exact signed-lane math and later a matched scalar kernel.
Verification	Ask which behavior the current coverage plan does not exercise.
Implementation	Identify the critical path and compare one controlled configuration change.
Release	Have a clean clone reproduce the documented claims.

Reading belongs immediately before the decision it informs. Preserve failed predictions as well as successful results.

Step 3: make a complete experiment entry

Use the template already in docs/EXPERIMENT_LOG.md. A useful entry lets another person repeat the observation.

```
Question and primary source:
Prediction and possible falsifier:
Git revision and RTL hashes:
Tool versions, target and seed:
Exact command and input program:
Expected result:
Observed result and raw artifact path:
Limitation, decision and next experiment:
```

Record the program words, not only a screenshot. Record the source revision with a seed. Record frequency constraint and target with an Fmax estimate. Keep source changes and report files together.

Do not replace the observation with a conclusion such as “works perfectly.” State what the test checked, what it observed and what remains outside its scope.

For a deliberate defect experiment, preserve the patch separately from the working branch. Explain why restoring the intended mechanism resolves that particular counterexample.

Step 4: control the comparison

Change one meaningful variable at a time. A DOT4 experiment should not simultaneously change memory size, target device, software workload and synthesis top.

Keep the original input and expected output fixed. Record any overhead included or excluded. For a scalar-versus-packed kernel, packing and memory traffic can matter as much as the arithmetic.

A repeated seed is useful only with the same relevant tool/source inputs. New versions may alter mapping, route quality or coverage instrumentation. Preserve tool identities and raw results.

State an inference explicitly. “The path report suggests forwarding selection limits timing” is different from a measured causal improvement after simplifying that path.

Use several route seeds to assess placement sensitivity when making comparative claims. Do not report only the best favorable run while hiding failures or worse results.

> **TRY IT BEFORE READING THE ANSWERS**
>
> 8B. Why does comparing different synthesis tops weaken an area claim even if both reports mention the same core?

Step 5: reproduce from a clean clone

Use a separate directory so generated local files cannot silently supply missing dependencies. The public repository is already available; use its actual URL.

```
git clone https://github.com/RahilBhavan/riscv.git riscv-check
cd riscv-check
docker build -t rv32-tools .
docker run --rm --user "$(id -u):$(id -g)" \
  -v "$PWD:/work" rv32-tools \
  make test lint coverage timing
```

Compare tool versions, RTL hashes and summary results. Raw logs can differ in runtime bookkeeping even when the important measurements reproduce.

The Dockerfile uses distribution packages rather than permanently pinned binary versions. For a lasting publication, archive the built image digest or an equivalent precise environment alongside the source.

Rebuilding these textbooks is separate: `make textbook-series` uses Node and Chromium on the host. The hardware-tools image does not promise the document renderer.

Step 6: write a README around reproducible claims

Lead with the implemented behavior and one command that runs it. Link architecture assumptions, source, verification and implementation reports beside the relevant claims.

For coverage, name the metric and numerator/denominator. For resources, name the device and boundary. For timing, distinguish requested constraint, achieved estimate and physical operation.

For this reference, the current figures are 252 passing programs, 11,924 checked retirement records, 37/38 core line instrumentation points and a 35.76 MHz median post-route estimate. These are not automatically your edited version's figures.

Keep limitations visible: combinational memory, local halt/trap behavior, no certification or formal integration, no physical board test, and a speedup measured on one kernel.

Do not write “95% test coverage, running at X MHz” when you measured a narrow line metric and a route estimate. The more precise description is more useful to a technical reader.

Step 7: publish the learning work

The repository already has a GitHub remote. Inspect it before pushing, then publish your learning branch with a descriptive commit.

```
git status --short
git diff --check
git remote -v
git add rtl tests docs reports
git commit -m "Explain the specific implemented change"
git push -u origin learning/build-core
```

Stage only files belonging to the intended change. Review generated reports and source hashes before committing. Do not add build caches, credentials or unrelated local notes.

The existing GitHub Actions workflow runs differential tests, lint, coverage and route, then uploads reports/logs. Inspect the run and artifacts after pushing. A remote repository with a stale README does not establish current results.

A pull request can make your changes reviewable before merging into main. Write the problem, resulting behavior and relevant validation for someone who has not seen this conversation.

Step 8: prepare a technical explanation

Use a short demonstration with a concrete program and checked outputs. Explain why a consumer needs forwarding, why a load creates a bubble, and how an EX redirect prevents a wrong-path store.

Calculate one mixed-sign DOT4 result by hand, then show the simulator's matching record. Explain which instruction fields select the custom operation and how the result enters ordinary forwarding.

Show a coverage report and its small denominator. Show the critical path and the hardware excluded from the implementation boundary. State one next experiment that could change your conclusion.

Distinguish the assistant-generated reference, the changes you wrote, the experiments you ran and the behavior you can explain. Your contribution can be specific and valuable without claiming every line as independent original work.

Practice answering a question by pointing to code or a trace, rather than repeating a headline metric.

Step 9: connect the artifact to research interests

Verify each recipient's full name and current interests before personalizing outreach. The qualified Michigan matches and primary profile links are in docs/RESEARCH.md.

For Das and Dreslinski, explain an introductory compute/architecture tradeoff and its measured limits. A four-byte dot unit is not in-memory computing, near-threshold silicon or a reproduction of either lab's accelerator work.

For Austin, a reproducible hazard counterexample and verification method are a more direct artifact. For Shahbaz and Chowdhury, reproducibility and systems evaluation are transferable skills, while a small core is an indirect research fit.

Link one public artifact and one specific observation. Do not claim a faculty member requested the design, that a lab endorsed it, or that a planned experiment already succeeded.

This volume prepares the evidence and explanation. It does not send emails or publish statements on your behalf.

A research experiment for this part

Write this entry before editing. It is a proposed experiment, not a result already obtained by the learner.

Question	Can another environment reproduce the public claims?
Prediction	A clean clone with the recorded tool image/source produces the same scoped summaries.
Experiment	Run the documented commands in a fresh directory, compare hashes and explain any changed results.
Evidence	Clone revision, environment identity, complete command log and linked reports.

What to record

Save source revision, tool versions, input words, seed, exact command, expected and observed behavior, raw artifact path and limitation. End with the decision that the observation supports.

Use `docs/EXPERIMENT_LOG.md`. If the result contradicts the prediction, keep the failure and explain the next test. Do not rewrite the prediction afterward to make the experiment appear successful.

Answers and completion check

8A. Example: on the same packed signed-byte inputs and checked outputs, does a DOT4 kernel reduce total cycles relative to a scalar RV32I software-multiply kernel after including packing and loop overhead? Then compare estimated runtime using each configuration's own route result.

8B. Different tops include different hardware, so the measured resource difference cannot be attributed solely to the feature.

Completion check

A clean clone reproduces the commands. Every headline has a linked report and scope. Your notes separate predictions, observations and unperformed work. You can explain one failure, fix and next test.

Sources and next steps

These sources support the concepts in this volume. The current RTL and local execution contract remain the authority for implementation-specific behavior.

1. Project source
2. Architectural tests
3. riscv-formal

Detailed chapter-mapped reading is in `docs/textbook/SOURCES.md`. Research-interest context is in `docs/RESEARCH.md`. The original overview and its interactive figures remain companion material.

Keep the evidence boundary

Teaching labs are separate from the 252-program baseline. Reference coverage and timing numbers belong to the recorded source/tool configuration. No physical board execution, ISA certification or formal proof is implied. The DOT4 speedup covers one kernel.

Continue with the next experiment identified in your own notes. Run the complete regression before treating a learning edit as a finished implementation.